FastAPI + SQLAlchemy + PostgreSQL

Authentication & Authorization

Complete Learning Workbook

A Project-Based, Theory-First Guide to Securing Backend APIs

Projects	6 Hands-on Projects
Stack	FastAPI, SQLAlchemy, PostgreSQL, Alembic
App	Todo App — grown project by project

Table of Contents

SECTION 0	The Big Picture: Full Roadmap	3
SECTION 1	Theory: Authentication vs Authorization	6
SECTION 2	Theory: Auth Methods & Industry Best Practices	8
PROJECT 1	Password Hashing & User Registration	14
PROJECT 2	JWT Authentication (Access + Refresh Tokens)	22
PROJECT 3	Role-Based Access Control (RBAC)	31
PROJECT 4	OAuth2 & Social Login (Google)	39
PROJECT 5	Rate Limiting & Brute Force Protection	46
PROJECT 6	API Key Authentication (Machine-to-Machine)	52
APPENDIX A	Security Risks & OWASP Top 10	58
APPENDIX B	Alembic Deep Dive Reference	62

Section 0 — The Big Picture: Full Roadmap

Before writing a single line of code, you need the **mental map** of everything you'll build. This section gives you the complete picture — what each project teaches, how they build on each other, and why the order matters.

#	Project	Core Question Answered
P1	**Password Hashing & User Registration**	Who are you? — Store credentials safely so we can verify identity later.
P2	**JWT Authentication (Access + Refresh)**	Prove it — Issue and validate cryptographic tokens so the server trusts every request.
P3	**Role-Based Access Control (RBAC)**	What can you do? — Attach roles/permissions so not every user can do everything.
P4	**OAuth2 / Social Login**	Let someone else verify you — Delegate authentication to Google/GitHub using industry-standard OAuth2.
P5	**Rate Limiting & Brute Force Protection**	How much can you do? — Guard the login door so attackers can't hammer it endlessly.
P6	**API Key Auth (Machine-to-Machine)**	Are you the right machine? — Service-to-service auth without user sessions.

How Projects Build on Each Other

Each project is **not isolated**. You will be **expanding the same Todo app** throughout the entire workbook. This simulates how real-world applications evolve, and you'll feel the architectural consequences of earlier decisions.

1. P1 creates the User model and password hashing — every other project needs this.
2. P2 uses the User model from P1 to issue tokens — authentication is now stateless.
3. P3 adds a Role model on top of P1+P2 — now we have authorization layers.
4. P4 adds OAuth2 as an alternative login path — users can now register via Google.
5. P5 wraps the P2 login endpoint with middleware guards — hardening what P2 built.
6. P6 is a parallel auth path (no sessions) — machine users authenticate differently to human users.

What You Will Know After Each Project

Milestone	Concepts Mastered
After P1	bcrypt, password salting, why plain-text storage is catastrophic, Alembic migrations for user tables
After P2	JWT structure (header.payload.sig), signing, expiry, refresh token rotation,

	stateless auth
After P3	RBAC vs ABAC, permission decorators, database-level role modelling, principle of least privilege
After P4	OAuth2 Authorization Code flow, PKCE, token exchange, FastAPI OAuth2PasswordBearer
After P5	Slowloris/brute-force attacks, Redis-backed rate limiting, account lockout strategies
After P6	Hashed API keys, scoped permissions, key rotation, machine-to-machine trust models

Authentication vs Authorization — The Core Distinction

The Single Most Important Distinction in Security

Authentication (AuthN) = *"Who are you?"* You prove your identity. The system verifies your claimed identity is real.

Authorization (AuthZ) = *"What are you allowed to do?"* After identity is proven, the system decides what you can access.

Think of a hotel: **authentication** is the front desk checking your ID and giving you a key card. **Authorization** is the key card only opening your room and the gym, not the penthouse suite.

In code: authentication is `get_current_user()`. Authorization is `require_role('admin')`. They always happen in that order.

The Authentication Decision Tree

When designing an application, ask these questions to decide which auth method to use:

Question	Recommended Approach
Is this human users via a browser/app?	Use JWT (Project 2) or OAuth2 Social Login (Project 4)
Are users coming from a specific trusted org?	Consider OAuth2 / SAML (enterprise SSO)
Is this machine-to-machine (server to server)?	Use API Keys (Project 6) or mTLS certificates
Do you need granular permissions?	Add RBAC on top (Project 3)
Is the login endpoint public-facing?	Add rate limiting (Project 5) regardless of auth type

Security Risks Preview — Why This Workbook Exists

Every project addresses at least one of these OWASP Top 10 risks. You'll learn the attack AND the defence:

OWASP Risk	Project	What Goes Wrong Without It
Broken Authentication	P1 + P2	Weak passwords, plain-text storage, no token expiry
Broken Access Control	P3	Users accessing other users' todos, missing role checks
Sensitive Data Exposure	P1 + P6	Storing passwords or API keys in plain text in the DB
Security Misconfiguration	P2 + P5	Weak JWT secrets, no HTTPS enforcement
Brute Force / DoS	P5	Unlimited login attempts, credential stuffing attacks
Insufficient Logging	All	No audit trails — can't detect or investigate breaches

Section 1 — Theory: Authentication vs Authorization Deep Dive

This section is **pure theory** — no code. Read it before any project. These concepts are the foundation every security decision rests on. Skipping theory leads to cargo-cult security: copy-pasting JWT code without understanding *why* it works.

1.1 Authentication in Depth

Authentication answers: "Is this entity who they claim to be?" There are three fundamental factors:

Factor	Examples	Trade-offs
Something you KNOW	Password, PIN, security question	Easy to implement, easy to steal (phishing, data breach)
Something you HAVE	OTP (TOTP/HOTP), hardware token, phone	Much harder to steal remotely, requires physical access
Something you ARE	Fingerprint, face, retina (biometrics)	Very strong but complex to implement, privacy concerns

Multi-Factor Authentication (MFA) combines two or more factors. Using only passwords (single factor) is now considered insufficient for any serious application. Projects 1 and 2 in this workbook build the password factor; MFA can be layered on top.

1.2 Authorization in Depth

Authorization answers: "Given I know who you are — what are you allowed to do?" There are three main models:

- **RBAC (Role-Based Access Control)** — Users have roles (admin, editor, viewer). Roles have permissions. Simple to manage, great for most applications. This is what Project 3 builds.
- **ABAC (Attribute-Based Access Control)** — Decisions based on attributes of the user, resource, and environment (e.g., 'user can access todo if user.department == todo.department'). More flexible but complex.
- **ACL (Access Control List)** — Per-resource permissions list (e.g., 'Alice can read this specific todo, Bob cannot'). Most granular, but hardest to manage at scale.

Why RBAC is the Industry Starting Point

Most production backends start with RBAC and add ABAC rules only where needed. This is because:

- RBAC maps naturally to real-world org structures (admin, moderator, user)
- Permission checks are simple: does this user's role include this permission?

- It's easy to audit: you can see exactly what each role can do in a single table
- Libraries and frameworks (FastAPI, Django, Rails) all have RBAC support

Reserve ABAC for fine-grained rules like *'user can only edit their own todos'* — which is a row-level ownership check, a common extension of RBAC.

1.3 The Request Lifecycle — Where Auth Happens

Understanding exactly where in the request lifecycle authentication and authorization fire is critical for debugging and design:

#	Stage	What Happens
1	**Client sends request**	HTTP request with Authorization header or Cookie
2	**Middleware intercepts**	FastAPI middleware or dependency extracts the token/key
3	**Authentication check**	Is the token valid? Is it expired? Does the user exist?
4	**User attached to request**	request.state.user or dependency injection sets current_user
5	**Authorization check**	Does current_user have permission for this specific endpoint/resource?
6	**Endpoint handler runs**	Business logic executes — only reached if all checks pass
7	**Response returned**	Data returned to the client

1.4 Stateful vs Stateless Authentication

This is one of the most important architectural decisions in auth design. Every JWT vs session debate comes down to this.

STATEFUL (Sessions)	Details
Storage	Server stores session data in DB/Redis. Client only holds a session ID.
Validation	On each request, server looks up session ID in storage — one DB query.
Revocation	Trivial: delete the session row. User is immediately logged out.
Scalability	Hard: all servers need access to the same session store. Requires Redis/sticky sessions.
Use when	You need instant revocation (banking, admin panels). Users are primarily web browser.

STATELESS (JWT)	Details
Storage	Server stores nothing. All data is in the token (JWT). Client holds the token.
Validation	On each request, server just verifies the token signature — no DB query.
Revocation	Hard: tokens are valid until expiry. Need a blacklist (defeats the purpose) or short expiry.

Scalability	Easy: any server can validate any token. No shared state needed. Perfect for microservices.
Use when	You need horizontal scalability. APIs consumed by mobile/SPA. Microservices architecture.

Industry Consensus (2025)

For most APIs: Use **short-lived JWTs (15 min)** for access tokens + **longer-lived refresh tokens (7-30 days)** stored in HttpOnly cookies. This gives you stateless scalability while still being able to 'revoke' via refresh token blacklisting.

For high-security: Add a Redis-backed token blacklist or use sessions. Banks, healthcare, government all use stateful sessions where immediate revocation is mandatory.

Section 2 — Auth Methods, Industry Best Practices & FastAPI

This section maps the entire landscape of authentication methods, when each is appropriate, and how FastAPI implements them. Read this before each project — it gives you the **why** behind every design decision.

2.1 The Complete Auth Methods Map

Method	How It Works	Use When	Watch Out For
HTTP Basic Auth	username:password base64 in every header	Internal tools, simple scripts, HTTPS required	Never use over HTTP. Password sent every request.
Session Cookies	Server stores session, client holds cookie	Traditional web apps, SSR, banking portals	Great for browsers. Does not work well for APIs/mobile.
JWT (Access Token)	Signed token with claims, sent in header	REST APIs, SPAs, mobile apps, microservices	Short expiry (15min). Stateless. Cannot revoke before expiry.
JWT (Refresh Token)	Long-lived token to get new access tokens	All apps using JWT access tokens	Store in HttpOnly cookie. Rotate on each use.
OAuth2	Delegated auth via external provider	Social login (Google, GitHub), enterprise SSO	Complex flow. Never implement from scratch — use a library.
API Keys	Long random key sent in header/query	Machine-to-machine, 3rd party API access	Hash before storing (like passwords). Support key rotation.
mTLS	Mutual TLS certificate auth	Service mesh, internal microservices, IoT	Very secure. Complex to manage certificates.
TOTP/MFA	Time-based one-time passwords	2nd factor for any human-facing auth method	Use as additional layer, not standalone.
Passkeys (WebAuthn)	Cryptographic challenge-response, no password	Modern web apps — passwordless future	Excellent UX. Requires WebAuthn library integration.

2.2 JWT Deep Dive — Structure, Signing, and Claims

A JWT is **not encrypted by default — it is only signed.** Anyone can base64-decode the payload and read its contents. Never put sensitive data (passwords, SSNs, payment info) inside a JWT. The signature only proves the token *wasn't tampered with*, not that the data is secret.

JWT Structure: Header.Payload.Signature

Header — algorithm and token type: `{"alg":"HS256","typ":"JWT"}`

Payload — claims (user data): `{"sub":"user_id","exp":1700000000,"role":"admin"}`

Signature — HMAC-SHA256(base64(header) + "." + base64(payload), SECRET_KEY)

All three parts are base64url-encoded and joined with dots. The signature uses your **SECRET_KEY** — if it leaks, attackers can forge any token.

Standard Claims you will use:

- `sub` — Subject — the user ID. Always use this for the user identifier.
- `exp` — Expiration — Unix timestamp. After this, the token is rejected.
- `iat` — Issued At — when the token was created. Used for rotation checks.
- `jti` — JWT ID — unique ID per token. Used to blacklist specific tokens.
- `nbf` — Not Before — token is invalid before this time. Rarely needed.
- `iss` — Issuer — your application domain. Used in multi-tenant systems.

HS256 vs RS256 — Which signing algorithm?

- **HS256** — Symmetric. Same secret key signs and verifies. Simple. Use when only one service issues and verifies tokens (monolith).
- **RS256** — Asymmetric. Private key signs, public key verifies. Use in microservices: only the auth service has the private key, but any service can verify tokens using the public key.
- **Best practice**: Use RS256 in production, HS256 is fine for learning.

2.3 Password Security — Why bcrypt?

Password storage is the #1 source of credential-related breaches. Understanding why bcrypt (or Argon2) is used requires understanding what attackers do with your database:

Attack Type	How It Works	Why bcrypt Defeats It
Dictionary Attack	Try common passwords and their variants	bcrypt: even 'password123' takes 100ms to hash — millions of guesses take years
Rainbow Table Attack	Pre-computed hash → password mappings	bcrypt: each hash includes a unique salt, making pre-computation useless
Brute Force	Try every possible combination	bcrypt: the work factor (cost) makes each attempt slow even with GPUs
Credential Stuffing	Use leaked password databases	bcrypt: breached hashes can't be reversed — leaked bcrypt hashes are useless

bcrypt vs Argon2 — What Should You Use?

bcrypt: Battle-tested (1999), universally supported in Python (passlib), excellent choice for any production app. Configurable cost factor.

Argon2: 2015 Password Hashing Competition winner. Memory-hard (harder to attack with FPGAs/ASICs). The academic/security community recommends it for new applications.

Industry reality: Both are fine. bcrypt is dominant in existing codebases. Argon2 is the

forward-looking choice. This workbook uses bcrypt for compatibility.

2.4 FastAPI's Auth Ecosystem

FastAPI has first-class support for security via its dependency injection system. Here are the key pieces:

- `OAuth2PasswordBearer` — FastAPI's built-in token extractor. Reads the Authorization: Bearer <token> header. Does NOT validate the token — just extracts it.
- `Security dependencies` — Inject get_current_user as a FastAPI Depends() — the endpoint never runs if auth fails. This is the idiomatic FastAPI pattern.
- `HTTPException 401/403` — 401 Unauthorized = not authenticated (no/invalid token). 403 Forbidden = authenticated but not authorized (wrong role).
- `Request state` — request.state.user stores the authenticated user for the duration of a request. Set in middleware, read in endpoints.
- `python-jose` — Python JWT library for encoding and decoding JWTs. Used in Projects 2 and 3.
- `passlib[bcrypt]` — Password hashing library. Handles bcrypt, salt generation, and constant-time comparison.

2.5 Industry Best Practices Checklist

This is what separates a production-ready auth system from a tutorial demo:

Practice	Standard
Password Storage	Hash with bcrypt (cost >= 12) or Argon2. Never MD5/SHA1/SHA256 alone.
JWT Secret	Minimum 256-bit random secret. Rotate it. Never commit to git.
Token Expiry	Access tokens: 15 minutes. Refresh tokens: 7-30 days.
HTTPS Only	Never transmit tokens over HTTP. Use HSTS headers.
Refresh Token Rotation	Issue a new refresh token on each use. Invalidate old one.
Rate Limiting	Limit login endpoint: 5 attempts / 15 minutes per IP.
Account Lockout	Lock account for 15-30 minutes after repeated failures.
Audit Logging	Log every auth event: login success, failure, token refresh, logout.
Secrets in Env Vars	Use .env files + pydantic-settings. Never hardcode secrets.
SQL Injection Prevention	Always use parameterized queries (SQLAlchemy ORM does this).
Timing Attacks	Use constant-time comparison for secrets (passlib handles this).
CORS	Restrict allowed origins. Don't use wildcard (*) in production.

PROJECT 1

Password Hashing & Secure User Registration

OWASP A02: Broken Authentication | Duration: ~3 hours

Project Overview

This is the **foundation of every other project** in this workbook. You cannot have authentication without first having a secure way to store and verify user credentials. By the end of this project, you will have a working `/auth/register` endpoint that properly hashes passwords before storing them.

What You Will Build

- A User model with hashed password storage in PostgreSQL
- A POST `/auth/register` endpoint that validates input and creates users
- An Alembic migration that adds the users table to your database
- A `security.py` module with password hashing and verification utilities
- A schema layer separating what the API accepts from what the DB stores
- A GET `/auth/me` stub endpoint to verify registration works

Security Risk This Project Addresses: OWASP A02 — Broken Authentication

The attack: An attacker breaches your database (via SQL injection, insider threat, or a misconfigured backup). Without hashing, they immediately have all user passwords. They can now: (1) log in as any user, (2) try those passwords on Gmail, banking apps, etc. (password reuse).

Why MD5/SHA1 are not enough: These are fast hashing algorithms. An attacker with a GPU can compute 10 billion MD5 hashes per second. With a database of bcrypt hashes, even with a GPU, they can only try ~100 per second per hash due to the intentionally slow bcrypt algorithm.

The defence you're building: bcrypt with a cost factor of 12. Each hash takes ~200ms to compute. That makes an offline dictionary attack take years instead of seconds.

Key Concepts to Understand Before Starting

Concept 1: What is Password Hashing (vs Encryption)?

Hashing is a one-way transformation: `password → hash`. You **cannot reverse it**. You can only verify by hashing the candidate password and comparing. **Encryption** is two-way: you can decrypt.

Never encrypt passwords — always hash them. If you ever need to retrieve the original password, that's a design flaw.

How bcrypt Verification Works (Why No Reverse Is Needed)

Registration: `hash = bcrypt.hash('mypassword123', rounds=12)` → Store hash in DB

Login: `bcrypt.verify('candidate_password', stored_hash)` → bcrypt hashes the candidate with the same salt embedded in the stored hash and compares.

The salt is stored **inside** the bcrypt hash string itself (the first 22 chars). You never store the salt separately.

Example hash: `$2b$12$EixZaYVK1fsbw1ZfbX3OXe.PmSNNy2jaBTaOtcxO6WJPPGpAXWTWa`

Breakdown: `$2b$` = bcrypt version, `12` = cost factor, next 22 chars = salt, remaining = hash

Concept 2: Why Alembic for Database Schema Changes?

When you add a new model in SQLAlchemy, the database table doesn't magically appear. You need to **tell the database to create it**. You have two options:

- **Base.metadata.create_all(engine)** — Creates all tables from scratch. Destroys existing data. Only valid for development/testing.
- **Alembic migration** — Tracks *what changed* and applies the minimum SQL needed. Preserves existing data. The only valid approach for any persistent database.

The Alembic Mental Model

Think of Alembic like Git for your database schema.

Each migration file is a 'commit' that describes exactly one schema change. Alembic tracks which migrations have run in the alembic_version table. Running 'alembic upgrade head' applies all un-run migrations in order.

This means: **any developer who clones your repo** and runs `alembic upgrade head` gets an identical database schema. No more 'it works on my machine' for schema.

Critical: migrations have **upgrade()** (apply change) and **downgrade()** (undo change). Always write the downgrade — you'll need to roll back broken deployments.

Concept 3: Pydantic Schemas vs SQLAlchemy Models

This is a very common confusion for beginners. You have **two different types of 'models'** in a FastAPI app:

Type	File	Purpose	Contains

SQLAlchemy Model	`models/auth.py`	Maps to a database table. Has columns, relationships. Used to read/write DB.	`class User(Base): id, email, hashed_password, ...`
Pydantic Schema	`schemas/auth.py`	Defines API contract. Validates input/output. Has nothing to do with DB.	`class UserCreate(BaseModel): email, password (plain text!)`

The Golden Rule: Never Expose hashed_password in API Responses

Your `UserCreate` schema accepts `password` (plain text). Your endpoint immediately hashes it.

Your `UserResponse` schema **never includes** `hashed_password`. It only returns `id, email, created_at`.

Your SQLAlchemy `User` model has `hashed_password` — but the Pydantic response schema filters it out. Pydantic's `model_config = ConfigDict(from_attributes=True)` handles the ORM-to-schema conversion.

Concept 4: Alembic's env.py — The Critical Connection

Alembic's `env.py` is the glue that connects Alembic to your database and your SQLAlchemy models. When you run `alembic revision --autogenerate`, Alembic looks at your models' metadata and *compares it to the current database schema* to generate the migration. If `env.py` doesn't import your models, Alembic won't know they exist.

The 3 Things env.py Must Do

1. Import your models: Add `from learn_auth.app.models import auth, todos` (or import Base). Without this, autogenerate won't see your models.

2. Set target_metadata: Set `target_metadata = Base.metadata`. This is the 'expected state'. Alembic diffs DB vs metadata to generate migrations.

3. Connect to the right database: Read `DATABASE_URL` from your config/env vars — not hardcoded. Use the same URL your app uses.

Workshop Steps — What to Build and Why

Each step is a self-contained task. **Do not look at the solution until you have genuinely attempted each step.** The struggle is where learning happens. After completing each step, run your tests and verify it works before moving on.

STEP 1 of 8 — Add Dependencies

What you will do: Add `passlib[bcrypt]` and `email-validator` to your project using `uv add`

Why: passlib is the industry-standard password hashing library for Python. It wraps bcrypt and handles salt generation, cost configuration, and constant-time comparison. email-validator is needed for Pydantic to validate email addresses.

Command: `uv add 'passlib[bcrypt]' email-validator`

What to verify: Both packages appear in `pyproject.toml` under [tool.uv.dependencies]

Pitfall: The `[bcrypt]` extra is mandatory — passlib without it will fall back to a slower pure-Python implementation.

STEP 2 of 8 — Update security.py

What you will do: Implement password hashing and verification functions in `src/learn_auth/app/core/security.py`

Why: Centralising security utilities in one module means: (1) any future change to hashing algorithm only touches one file, (2) endpoints never import passlib directly, only the abstraction.

Functions to implement:

- `get_password_hash(password: str) -> str` — takes plain-text password, returns bcrypt hash
- `verify_password(plain_password: str, hashed_password: str) -> bool` — constant-time comparison

Why constant-time comparison matters: A naive `hash1 == hash2` comparison returns early on the first mismatched character, leaking timing information. passlib's `verify()` always takes the same time regardless of how wrong the password is.

What to verify: Write a quick test in a Python REPL: hash a password, verify with correct and wrong passwords.

STEP 3 of 8 — Create the User SQLAlchemy Model

What you will do: Add a `User` class to `src/learn_auth/app/models/auth.py`

Fields to include:

- `id: Integer, primary_key=True` — surrogate key. Use Integer not UUID for simplicity now; we'll discuss UUID trade-offs.
- `email: String(255), unique=True, nullable=False` — must be unique; users

identify themselves by email

- `hashed_password: String(255), nullable=False` — bcrypt outputs 60-char strings, 255 gives headroom
- `is_active: Boolean, default=True` — allows disabling accounts without deleting them
- `is_superuser: Boolean, default=False` — quick admin flag (replace with full RBAC in Project 3)
- `created_at: DateTime, default=func.now()` — audit trail. Always have timestamps on user records

Why is_active instead of deleting: Deleting users causes foreign key issues if they own todos. is_active=False is a 'soft delete'. Endpoints check this flag before allowing operations.

Relationship to add: Add `todos: relationship('Todo', back_populates='owner')` so User knows about their todos. This is a one-to-many: one user, many todos.

STEP 4 of 8 — Create Pydantic Schemas for Users

What you will do: Create `src/learn_auth/app/schemas/auth.py` with user-related Pydantic models

Schemas to create:

- `UserBase` — shared fields: email (EmailStr). Base class for others.
- `UserCreate(UserBase)` — adds: password (str). Has field validators: min 8 chars, require uppercase + digit.
- `UserResponse(UserBase)` — adds: id, is_active, created_at. Has `model_config = ConfigDict(from_attributes=True)`. **No password field!**
- `UserInDB(UserBase)` — internal use only: adds hashed_password. Never returned by endpoints.

Why from_attributes=True: By default Pydantic works with plain dicts. Setting `from_attributes=True` tells Pydantic to read attributes from ORM objects, not just dict keys. Without it, `UserResponse.model_validate(db_user)` will fail.

Why separate UserCreate and UserResponse: These represent what the API *accepts* vs what it *returns*. They evolve independently. Having one 'User' schema that does both leads to bugs: you'll accidentally expose hashed_password or accept fields that shouldn't be writable.

Password validation rule: Add a Pydantic `@field_validator('password')` that enforces: minimum 8 characters, at least one uppercase letter, at least one digit. Return clear error messages.

STEP 5 of 8 — Create the Alembic Migration

What you will do: Generate and run an Alembic migration that adds the users table to PostgreSQL

First, fix env.py to import your models:

- Open `alembic/env.py`
- Find the `target_metadata` line
- Add the import: `from learn_auth.app.models.auth import User  # noqa: F401`
- Ensure `target_metadata = Base.metadata` is set

Why the noqa comment: Linters (ruff) will flag this import as 'unused' because the variable `User` is never referenced in env.py. But the import is essential: it registers the User model with Base.metadata as a side effect. The noqa comment silences the false positive.

Generate the migration:

- `uv run alembic revision --autogenerate -m 'add_users_table'`
- Open the generated file in `alembic/versions/` and **read every line**. Understand the `upgrade()` and `downgrade()` functions.
- Verify it creates: `users` table with correct columns, `unique constraint` on email, `index` on email

Apply the migration:

- `uv run alembic upgrade head`
- Verify in psql: `\d users` should show the table schema

Key learning: Never manually edit the DB schema with SQL. Always use Alembic. This keeps your schema version-controlled and reproducible.

STEP 6 of 8 — Create the User Service

What you will do: Create `src/learn_auth/app/services/auth.py` with user registration logic

Functions to implement:

- `get_user_by_email(db: Session, email: str) -> User | None` — DB lookup by email

- `create_user(db: Session, user_in: UserCreate) -> User` — validates uniqueness, hashes password, saves to DB

Why a service layer? The **Endpoints → Services → Database** pattern separates HTTP concerns (request/response) from business logic (what to do). Your endpoint should be 3-5 lines. Your service contains the actual logic. Benefits:

- Endpoints stay thin and readable
- Services are testable without HTTP machinery
- Logic can be reused by multiple endpoints (e.g., admin user creation reuses the same service)

The create_user flow:

7. Look up email — if exists, raise a specific `ValueError('Email already registered')`
8. Hash the password: `hashed = get_password_hash(user_in.password)`
9. Create the ORM object: `db_user = User(email=..., hashed_password=hashed)`
10. Add and commit: `db.add(db_user); db.commit(); db.refresh(db_user)`
11. Return `db_user` (the service returns an ORM object, not a schema)

Why db.refresh(): After a commit, SQLAlchemy expunges the object to keep its state clean. `refresh()` re-fetches it from the DB, so `db_user.id` and `db_user.created_at` are populated.

STEP 7 of 8 — Create the Auth Router and Register Endpoint

What you will do: Create `src/learn_auth/app/api/v1/endpoints/auth.py` and wire up the register endpoint

Endpoint to implement:

- `POST /auth/register` — accepts UserCreate, returns UserResponse, status 201

The endpoint function logic:

12. Accept `user_in: UserCreate` from request body (FastAPI handles parsing + validation)
13. Inject `db: Session = Depends(get_db)` for the database session
14. Call `create_user(db, user_in)` — catch ValueError and convert to HTTP 400
15. Return `UserResponse.model_validate(db_user)`

Error handling pattern: Catch `ValueError` from the service and re-raise as `HTTPException(status_code=400, detail=str(e))`. Services raise Python exceptions; endpoints convert them to HTTP responses. This separation makes services testable.

Why 201 not 200: HTTP 200 means 'OK'. HTTP 201 means 'Created'. Use the correct status code — it's part of your API contract. FastAPI: `@router.post('/register', response_model=UserResponse, status_code=201)`

Register the router in routers.py:

- Import the auth router: `from .endpoints import auth`
- Add: `api_router.include_router(auth.router, prefix='/auth', tags=['auth'])`

STEP 8 of 8 — Update the Todo Model and Test Everything

What you will do: Add `user_id` foreign key to the Todo model, create a migration, and manually test the full flow

Update models/todos.py:

- Add `user_id: Integer, ForeignKey('users.id'), nullable=False`
- Add `owner: relationship('User', back_populates='todos')`

Why nullable=False: Every todo must belong to a user. A todo without an owner is a data integrity violation. The NOT NULL constraint enforces this at the database level — not just in application code. Database constraints are your last line of defense.

Generate and run the migration:

- `uv run alembic revision --autogenerate -m 'add_user_id_to_todos'`
- Inspect the migration: it should add `user_id` column and the foreign key constraint
- `uv run alembic upgrade head`

Test with curl or HTTPie:

- `POST /auth/register` with `{"email":"test@example.com","password":"SecurePass123"}` — expect 201
- POST same email again — expect 400 with 'Email already registered'
- POST with weak password `"password"` — expect 422 Unprocessable Entity with validation error details
- Check database: `SELECT email, hashed_password FROM users;` — verify hash starts with `$2b$12$`

What to verify in the DB: The `hashed_password` column should contain a bcrypt hash, not the plain password. If you see the plain password — something is wrong in your flow.

What You Learned in Project 1

Concept	What You Now Understand
bcrypt & passlib	Why MD5/SHA1 fail. How bcrypt's salt and cost factor defeat offline attacks.
One-way hashing	The fundamental difference between hashing and encryption for credentials.
Pydantic schema separation	UserCreate vs UserResponse — accepting vs exposing data.
SQLAlchemy relationships	User → Todo one-to-many with foreign key and back_populates.
Alembic migrations (why)	Schema migrations as version control. Never touch DB schema manually.
env.py import pattern	Why models must be imported in env.py for autogenerate to work.
Service layer pattern	Endpoints → Services separation. Services raise Python errors, endpoints convert to HTTP.
Constant-time comparison	Why direct string comparison of secrets leaks timing information.
Database constraints	NOT NULL, UNIQUE at DB level as the last line of defense.

What's Missing — and What Project 2 Will Add

Current state: Users can register, passwords are secure. But the app can't tell *which* user is making a request. Every endpoint is still anonymous.

The problem: If you call GET /todos, should you return *all* todos or just the calling user's todos? Right now, you have no way to know who's calling.

What Project 2 adds: A POST /auth/login endpoint that verifies credentials and returns a JWT. All subsequent requests include this JWT, and the server extracts the user identity from it. Endpoints can then filter todos by owner.

The architectural shift: From 'anyone can do anything' to 'only authenticated users can see their own data'. This is the core of user-scoped APIs.

PROJECT 2

JWT Authentication: Access & Refresh Tokens

Coming Next | Ask for Project 2 explanation when ready

Preview: In Project 2, you will implement the full JWT login flow. Users will POST their credentials to /auth/login and receive two tokens: a short-lived access token (15 min) and a long-lived refresh token (7 days). You'll understand exactly how FastAPI's OAuth2PasswordBearer works, how token payloads are structured, and why refresh tokens exist. You'll also add the get_current_user dependency that locks down all Todo endpoints.

- JWT encoding with python-jose
- OAuth2PasswordBearer in FastAPI
- access_token + refresh_token pattern
- Protecting endpoints with Depends(get_current_user)
- Token refresh endpoint
- Understanding 401 vs 403 correctly

PROJECT 3

Role-Based Access Control (RBAC)

Coming Later | Ask when you reach this project

Preview: Project 3 adds a roles system. You'll create Role and Permission models, add a many-to-many user-role relationship, and build reusable require_role() and require_permission() dependencies. You'll understand the difference between user owns this resource (row-level authorization) vs user has admin role (role-level authorization).

PROJECT 4

OAuth2 & Social Login (Google)

Coming Later | Ask when you reach this project

Preview: Project 4 teaches you the OAuth2 Authorization Code Flow with PKCE. You'll add 'Login with Google' to your Todo app — handling the redirect, code exchange, and user upsert. This is the flow behind every 'Sign in with Google/GitHub' button on the internet.

PROJECT 5

Rate Limiting & Brute Force Protection

Coming Later | Ask when you reach this project

Preview: Project 5 adds rate limiting to the login endpoint using Redis (or an in-memory store). You'll simulate a brute force attack to see it working, implement account lockout, and understand IP-level vs account-level throttling.

PROJECT 6

API Key Authentication (Machine-to-Machine)

Coming Later | Ask when you reach this project

Preview: Project 6 adds a separate auth path for machine-to-machine communication. You'll generate hashed API keys, implement the X-API-Key header validation, add key scoping (read-only vs read-write keys), and learn why you hash API keys just like passwords.

Appendix A — Security Risks & OWASP Top 10

The OWASP Top 10 is the industry standard reference for web application security risks. Here's how each risk maps to your FastAPI application:

Risk	P#	What Goes Wrong	Defence
A01: Broken Access Control	P3	Todos accessible across users; missing admin checks	RBAC with row-level ownership checks
A02: Broken Authentication	P1+P2	Plain-text passwords; no token expiry; no rate limit	bcrypt + JWT + rate limiting
A03: Injection	All	SQL injection via raw string queries	SQLAlchemy ORM parameterizes all queries
A04: Insecure Design	P3+P4	No security requirements defined upfront	RBAC design before implementation
A05: Security Misconfiguration	P2	Default secrets; debug mode in production; open CORS	Env var secrets; strict CORS; no DEBUG in prod
A06: Vulnerable Components	All	Outdated deps with CVEs	uv lock + dependabot + periodic audit
A07: Identity/Auth Failures	P2+P5	No lockout; weak JWT secret; token not expired	Rate limiting + strong secrets + short expiry
A08: Data Integrity Failures	P2	JWT algorithm confusion attacks	Explicitly specify algorithm; use RS256 in prod
A09: Logging Failures	All	No audit log for auth events	Log login/logout/failures to structured log
A10: SSRF	P4	OAuth redirect to internal services	Validate redirect_uri against whitelist

Appendix B — Alembic Deep Dive Reference

This appendix is your go-to reference for Alembic whenever you get stuck. It covers the most common operations and pitfalls.

B.1 The Commands You Will Use Every Day

- `alembic upgrade head` — Apply all pending migrations. Run after any new migration.
- `alembic downgrade -1` — Undo the last migration. Use to fix a bad migration.
- `alembic revision --autogenerate -m 'desc'` — Generate a migration from model changes.
- `alembic current` — Show which migration version the DB is at.
- `alembic history` — List all migrations in order.
- `alembic stamp head` — Mark current DB as at head WITHOUT running migrations. Use when DB already matches models.

B.2 Common Alembic Pitfalls

Pitfall	Explanation
Model not detected	Forgot to import the model in env.py. Autogenerate can only see models that are imported before it reads metadata.
'Nothing to migrate' when there are changes	target_metadata is set to the wrong Base. Make sure all models inherit from the same Base.
Multiple heads	Two developers created migrations from the same head. Run alembic merge heads to create a merge migration.
Can't downgrade past initial	Your first migration's downgrade() should drop all tables it created — make sure it's complete.
Autogenerate misses some changes	Alembic can't detect: stored procedures, CHECK constraints (in older versions), computed columns. Write these manually.

B.3 The env.py Checklist

Every time you create a new model, verify these 3 things in env.py:

16. New model is imported (directly or via its module)
17. target_metadata = Base.metadata is using the correct Base (same Base all models inherit from)
18. DATABASE_URL is being read from environment variables, not hardcoded